

Hemut Trial Project: Real-Time Logistics Collaboration Platform

Build a Slack-style collaboration platform for a logistics company: channels, DMs, real-time messaging, and one AI feature of your choosing. The where and how of AI integration is intentionally open-ended.

Timeline & Submission:

- **Deadline: 72 hours from when this assignment is sent.**
- Reply with: public GitHub repo link, optional deployed URL, 3–5 min Loom walkthrough (required).
- Commit frequently. If something blocks you, document it in the README and ship a workaround.

Frontend (Next.js or React Native):

Required screens (each must be reachable and functional):

- Login / Register page with DB-backed credential validation.
- Channel list / sidebar with joined channels (e.g. #route-east, #warehouse-mumbai) and unread indicators.
- Channel message view with real-time messages, sender, and timestamp.
- Direct message (1:1) view.
- One logistics-context surface: shipment preview card on a message, OR /shipment <id> slash command, OR shipments sidebar.

Other requirements:

- Online / offline / away presence indicators.
- Form validation using raw XMLHttpRequest (NOT fetch or axios).

Why XHR? We want to see you handle HTTP request lifecycle, async, and progress/abort/timeout/error events directly. Modern libraries hide all of this. This is the only tooling constraint in the assignment.

Backend (FastAPI):

- REST or gRPC endpoints: register/login, create/list/join/leave channels, post messages (channel + DM), fetch history with pagination, mock shipment lookup by ID.
- WebSockets for pushing new messages, presence changes, and AI responses.
- Optional: Webhooks to ping an external service when a shipment-tagged message is posted.

AI Integration (Open-Ended, Candidate's Call):

Implement one well-executed AI feature and justify the choice in the README. We are deliberately NOT prescribing this. Possible directions:

- Thread summarization ("Catch me up on #route-east from the last 24 hours").
- RAG-based Q&A; over chat history, BOLs, or shipment docs.

- Entity extraction (PO numbers, tracking IDs, ETAs, addresses).
- Delay / escalation detection that surfaces flagged messages to managers.
- Inline translation, action suggestions, or your own idea.

README must answer: why this feature, how it was implemented, and what would change in production.

Infrastructure:

- PostgreSQL is required as the primary datastore. In-memory storage (Python dicts, lists, etc.) is NOT acceptable.
- Redis is required for WebSocket pub/sub fan-out across workers and for lightweight caching (presence, rate-limit counters).
- Required tables: users, channels, memberships, messages, shipments (mock). Anything else you come up with.
- Deploy Postgres + Redis via Docker locally, or RDS/ElastiCache on AWS.
- Use .env for config (DB URL, Redis URL, JWT secret, LLM API keys).
- Push code to a public GitHub repository and share the link.

Testing (Required):

- Basic unit or integration tests for backend API endpoints (auth, channel ops, messaging).
- At least one test exercising the AI feature with a mocked LLM response (so CI is deterministic and not billable).
- Frontend tests encouraged but not required.

Bonus (does not affect pass/fail, but can lift to Strong):

Not required. Missing all of these will not fail a submission, but they can lift a borderline submission to Strong, especially when grounded in the logistics use case.

- Message search, threaded replies, file attachments, notification badges, Docker setup.
- A second AI feature, or a creative extension of the chosen one.
- Your own major creative addition.

Requirement	Tech Used
FastAPI	Backend API
Next.js or React Native	Frontend
PostgreSQL	Primary datastore (required)
Redis	WebSocket pub/sub + caching (required)
WebSockets / Webhooks / gRPC	Realtime & integrations
Git / GitHub	Source control
LangChain / OpenAI / Anthropic / Pipecat	AI feature (candidate's choice)

AWS	Optional deployment
-----	---------------------

Evaluation Rubric

Every criterion below must reach at least **"Passes"** to move forward. A submission is **Strong** when 5+ criteria reach "Strong" with no criterion at "Does Not Pass".

Criterion	Does Not Pass	Passes	Strong
Core Chat	Messages do not reliably send/receive in real time; auth, channels, or DMs broken.	Register, login, channels, DMs, real-time delivery, and presence all work end-to-end.	Robust under reconnects, correct message ordering on rejoin, graceful error states.
Postgres + Redis	In-memory used as primary store, or Redis missing.	Postgres holds durable data with a sensible schema. Redis used for pub/sub or caching.	Indexed schema with clean migrations. Redis used for pub/sub AND caching with clear separation.
AI Feature	No AI feature, or it does not work end-to-end.	One AI feature works end-to-end with basic README justification.	Well-chosen for real user pain; handles streaming/sources/fallbacks; README explains tradeoffs.
Code Quality	Disorganized, mixed concerns, copy-pasted.	Reasonable separation of concerns, readable structure, idiomatic.	Clean architecture, easy to extend, consistent style.
Real-Time Correctness	Messages drop, duplicate, or arrive out of order in normal use.	Messages arrive in order under normal conditions; presence works.	Handles disconnects and replays missed messages; lifecycle managed across navigation.
Testing	No tests, or tests do not run.	Happy-path tests for backend; AI feature tested with mocked LLM.	Meaningful coverage including failure paths; tests are well-structured and fast.
Documentation	README missing or unclear on how to run the project.	Clear setup, architecture overview, and AI feature write-up.	Above + thoughtful tradeoffs section, diagrams, and meaningful commit history.
Logistics Context	Generic chat clone with no domain awareness.	At least one logistics-grounded touch (shipment refs, slash command, sidebar).	Domain awareness throughout: channel naming, mock data, UI affordances, and the AI feature.

Conceptual Questions

Sample of the kinds of questions we will explore in follow-up interviews. Candidates do not need to answer these in the submission.

Architecture & Real-Time

- **How would you design this system to handle 10,000+ concurrent users across many channels in real time?**

WebSockets with channel-scoped pub/sub via Redis, async FastAPI handlers, horizontal scaling behind a sticky-session load balancer, and client-side batching to reduce render churn.

- **Why is Redis required alongside PostgreSQL in this architecture?**

Postgres gives durable storage for messages and users. Redis solves what Postgres can't: when WebSocket connections are spread across multiple workers, a message posted on worker A must reach users connected to worker B. Redis pub/sub broadcasts the event so all workers fan it out. It is also a natural fit for presence and rate-limit counters.

- **If drivers in low-connectivity areas drop off frequently, how do you handle message delivery?**

Persist messages with monotonic IDs, let clients request "messages since ID X" on reconnect, and use heartbeats with exponential-backoff reconnection.

AI & Product Thinking

- **Where in this product would AI create the most value, and why?**

Open-ended. Strong answers tie the feature to a real user pain (e.g. dispatchers losing hours catching up on overnight threads → summarization) rather than chasing novelty. Bonus for cost, latency, and trust considerations.

- **What are the failure modes of LLM answers in a logistics context, and how would you mitigate them?**

Hallucinated tracking numbers, wrong ETAs, fabricated shipment status. Mitigate by grounding outputs in retrieved sources, citing source messages, refusing out-of-context queries, and clearly labeling AI output.

Security & Frontend

- **How would you protect admin-only actions (creating channels, removing members)?**

JWT with role-based claims; FastAPI dependency checks role before executing. Audit-log admin actions server-side.

- **What vulnerabilities arise in a multi-user chat product, and how do you prevent them?**

XSS (escape on render), CSRF (token-based protection), message spoofing (server derives sender from auth, never trusts client), workspace tenancy leaks (scope every query), and prompt injection in AI features (treat retrieved content as untrusted data, not instructions).

- **What are the hardest parts of managing real-time chat state in React?**

Avoiding stale closures in WebSocket handlers, preventing memory leaks on unmount, reconnecting cleanly after network loss, ordering interleaved arrivals, and keeping re-renders cheap under load.